

Explicación Profunda del Proyecto

Arquitectura, Tecnologías y Cómo Funciona

Curso 2025-26 · CESUR · DAM

Alejandro Fernández Morales

Tabla de Contenidos

- **1. Introducción** — Qué es ReténDigital y por qué se hizo
- **2. La Estructura General del Proyecto** — Carpetas, archivos y organización
- **3. ¿Qué es Android?** — La plataforma donde corre la app
- **4. Las Tecnologías Principales (4 pilares)** — Room, Hilt, Retrofit, Google Maps
- **5. Arquitectura MVVM Explicada** — Cómo se organizó el código
- **6. La Base de Datos — Room en Profundidad** — Cómo se guardan los datos
- **7. Inyección de Dependencias — Hilt** — Cómo se conectan las piezas
- **8. Las 5 Pantallas de la App** — Qué hace cada una
- **9. El Flujo Completo — De principio a fin** — Qué pasa cuando abres la app
- **10. Preguntas Frecuentes sobre el Código** — Respuestas a dudas comunes

1. INTRODUCCIÓN · ¿Qué es ReténDigital?

ReténDigital es una **aplicación móvil Android** que te ayuda a gestionar el día a día de un retén contraincendios. Imagina que eres jefe de equipo en un retén. Antes, todo se hacía con papel:

- Cuadrante de turnos apuntado en una hoja
- Estado de vehículos en otra hoja
- Inventario en un carpeta aparte
- Avisos de intervenciones en un grupo de WhatsApp

El problema: si alguien pregunta "¿dónde está el cuadrante?", pierdes 10 minutos buscando entre papeles. Si la app se cae y pierdes datos... es un caos.

La solución ReténDigital: todo en un móvil. Abres la app y tienes acceso a los turnos, los vehículos, el inventario y un registro de intervenciones con GPS.

Dato importante: Sin internet

La app funciona **completamente sin internet**. Los datos se guardan en la base de datos del propio móvil. Así, si estás en mitad del campo sin señal (que es lo normal en incendios forestales), la app sigue funcionando. Eso la hace única: es offline-first.

2. LA ESTRUCTURA DEL PROYECTO · Las Carpetas y Archivos

Un proyecto Android tiene muchas carpetas. La mayoría las genera Android Studio automáticamente. Las que **TÚ** creaste y modificaste son estas:

```
RetnDigital/ └─ app/ │ └─ src/main/ │ │ └─ java/com/alejandro/
retendigital/ │ │ │ └─ ui/ ← LAS PANTALLAS │ │ │ │ └─ MainActivity │ │ │
│ │ │ └─ MenuPrincipalActivity │ │ │ │ └─ VehiculoListActivity │ │ │ │ └─
CuadranteActivity │ │ │ │ └─ InventarioActivity │ │ │ │ └─
IntervencionActivity │ │ │ └─ data/ ← LA BASE DE DATOS (ROOM) │ │ │ │ └─
AppDatabase │ │ │ │ └─ VehiculoEntity │ │ │ │ └─ VehiculoDao │ │ │ │ └─
VehiculoRepository │ │ │ └─ di/ ← INYECCIÓN (HILT) │ │ │ │ └─ AppModule
│ │ │ └─ RetenDigitalApp │ │ └─ res/layout/ ← LOS DISEÑOS (XML) │ │ └─
activity_main.xml │ │ └─ activity_menu_principal.xml │ │ └─
activity_vehiculo_list.xml │ │ └─ ... (más XMLs) │ └─ build.gradle ← LAS
LIBRERÍAS (ROOM, HILT, etc) └─ AndroidManifest.xml ← LA RECETA DE LA APP
```

La estructura es **separación de responsabilidades**:

- **ui/**: son las Activities, lo que ve el usuario (las pantallas)
- **data/**: es la base de datos, cómo guardamos información
- **di/**: es la inyección de dependencias, cómo conectamos las piezas
- **res/layout/**: son los diseños visuales (botones, campos de texto, etc)

3. ¿QUÉ ES ANDROID? · La Plataforma Base

Android es el **sistema operativo** que corre en tu móvil Samsung, Xiaomi, Motorola, etc. (no en iPhone, que usa iOS).

Una **app Android** es como una aplicación de Windows que corre en tu PC, pero en el móvil. Está escrita en Java (o Kotlin) y se comunica con el sistema operativo.

El ciclo de vida de una Activity (la pantalla)

Cada pantalla (Activity) en Android tiene un "ciclo de vida":

1. **onCreate()**: Se crea la pantalla. Aquí inicializamos botones, layouts, etc.
2. **onStart()**: La pantalla es visible
3. **onResume()**: El usuario puede interactuar
4. **onPause()**: El usuario se va a otra pantalla
5. **onStop()**: La pantalla está oculta
6. **onDestroy()**: Se destruye la pantalla

En tu código, casi todo lo haces en **onCreate()**. Ahí es donde dices: "Botón GPS, cuando te pulsen, haz esto".

Las **Activities** son las pantallas. En tu app tienes 6:

- **MainActivity** (Login): La primera pantalla
- **MenuPrincipalActivity**: El menú con 4 botones
- **VehiculoListActivity**: Lista de vehículos
- **CuadranteActivity**: Calendario con turnos
- **InventarioActivity**: Lista de material
- **IntervencionActivity**: Registro de intervenciones con mapa

4. LAS 4 TECNOLOGÍAS PRINCIPALES · Los Pilares de tu App

Tu app usa 4 librerías (piezas de código) importantes. Cada una hace una cosa específica:

4.1 ROOM — La Base de Datos

¿Qué es? Room es una librería que facilita trabajar con bases de datos SQLite en Android. SQLite es una base de datos muy pequeña que se instala en el propio móvil, sin necesidad de servidor.

Analogía: Un cuaderno digital

Imagina que quieres anotar la flota de vehículos. Antes lo hacías en papel. Ahora lo haces en un "cuaderno digital" guardado en el móvil. Room es quien maneja ese cuaderno.

Antes: "Dosis, apunta en el papel: Volvo XC90, matrícula ZX-123"

Ahora: "Room, guarda en la BD: Volvo XC90, ZX-123"

Room tiene tres componentes clave en tu código:

- **VehiculoEntity:** La "forma" de un vehículo (qué datos tiene)
- **VehiculoDao:** El "mando a distancia" para operar la BD (insert, delete, update)
- **AppDatabase:** La base de datos misma

4.2 HILT — Inyección de Dependencias

¿Qué es? Hilt es una librería que automáticamente "conecta" las piezas de tu código. Sin Hilt, tendrías que crear objetos a mano todo el tiempo. Con Hilt, él lo hace por ti.

Analogía: El distribuidor de correo

Imagina que cada parte de tu app es una oficina. La BD es una oficina, el Repository otra, etc. Sin Hilt, cada oficina tendría que ir a buscar la información que necesita de otras oficinas — caos total.

Con Hilt, hay un "distribuidor de correo" que lleva la información a cada oficina automáticamente.

En tu app, Hilt hace esto:

```
@HiltAndroidApp ← Le dices a Hilt que arranque public class
RetenDigitalApp extends Application { } Luego en VehiculoRepository:
@Inject VehiculoDao dao; ← Hilt automáticamente proporciona el DAO
```

4.3 RETROFIT — Llamadas a APIs Remotas

¿Qué es? Retrofit es una librería para hacer peticiones HTTP a servidores remotos. Por ejemplo, consultar el tiempo en una API.

Estado en tu código: Retrofit está **añadido en las dependencias** (build.gradle), pero **no está implementado**. Lo dejaste preparado para versiones futuras (consultar Open-Meteo para el clima).

4.4 GOOGLE MAPS SDK — Mapas y Ubicación

¿Qué es? Google Maps SDK es la librería que te permite incrustar mapas dentro de tu app y trabajar con coordenadas GPS.

En tu IntervencionActivity, lo usas para mostrar un mapa satélite de la zona de Aragón. Cuando el usuario pulsa "Capturar GPS", el botón genera coordenadas UTM (el formato que usan los bomberos).

¿Qué es UTM?

UTM es un sistema de coordenadas usado por emergencias. En lugar de "latitud 41.6488, longitud -0.8891" (GPS normal), dice "Huso 30T, X: 675200, Y: 4613500".

Aragón está en el Huso 30T. Tu app genera coordenadas ficticias de ejemplo de Zaragoza cuando el usuario pulsa el botón.

5. ARQUITECTURA MVVM · Cómo Está Organizado el Código

MVVM significa **Model-View-ViewModel**. Es una forma de organizar el código para que sea más fácil de mantener.

```
|-----| VISTA (View) | | - MainActivity
| | - MenuPrincipalActivity | | - activity_main.xml (diseño) | | | (Lo
que ve el usuario) |-----| | |
(comunica) ↓ |-----| LÓGICA (ViewModel)
| | - Procesa datos | | - Prepara info para mostrar | | - Reacciona a
clicks de botones | | | (El "intermediario") |
|-----| | | (pide datos) ↓
|-----| DATOS (Model) | | - AppDatabase
| | - VehiculoRepository | | - VehiculoDao | | - VehiculoEntity | | |
(Donde se guardan los datos) |-----|
```

La ventaja: si mañana quieres cambiar el diseño de una pantalla (cambiar botones de lugar), no tocas la base de datos ni la lógica. Solo cambias el XML de la vista.

En tu app, esto se ve así:

- **V (View)**: MainActivity, VehiculoListActivity, etc.
- **VM (ViewModel)**: La lógica dentro de onCreate() de cada Activity
- **M (Model)**: VehiculoRepository, VehiculoDao, AppDatabase

Tu código **no usa ViewModel formal** (que sería una clase extra), pero sigue el patrón: las Activities tienen la lógica, y llaman al Repository para pedir datos.

6. ROOM EN PROFUNDIDAD · Cómo Guardas los Datos

Room tiene 4 componentes. Vamos a explicarlos como si la BD fuera un restaurante:

6.1 VehiculoEntity — La "Forma" de un Vehículo

Entity es como la "plantilla" de un vehículo. Define qué datos tiene un vehículo en la BD:

```
@Entity(tableName = "tabla_vehiculos") public class VehiculoEntity
{ @PrimaryKey(autoGenerate = true) public int id; // ID único (1, 2, 3...)
  @ColumnInfo(name = "matricula") public String matricula; // Ej: "ZX-123"
  @ColumnInfo(name = "marca") public String marca; // Ej: "Volvo"
  @ColumnInfo(name = "modelo") public String modelo; // Ej: "XC90"
  @ColumnInfo(name = "tipo_vehiculo") public String tipo; // Ej:
  "BUP" (Bomba Urbana) @ColumnInfo(name = "estado") public String estado; //
  Ej: "Operativo" }
```

Cada vehículo es una **fila** en la tabla. Las columnas son: id, matricula, marca, modelo, tipo, estado.

6.2 VehiculoDao — El "Mando a Distancia"

DAO = Data Access Object. Es la interfaz que te permite hacer operaciones en la BD. Hay 4 operaciones básicas (CRUD: Create, Read, Update, Delete):

```
@Dao public interface VehiculoDao { @Query("SELECT * FROM
tabla_vehiculos") LiveData< getAllVehiculos(); // LEER @Insert void
insert(VehiculoEntity vehiculo); // CREAR @Update void
update(VehiculoEntity vehiculo); // ACTUALIZAR @Delete void
delete(VehiculoEntity vehiculo); // BORRAR }
```

¿Qué es LiveData?

LiveData es un contenedor de datos que "avisa" a la pantalla cuando hay cambios. Si alguien añade un vehículo nuevo a la BD, el LiveData

automáticamente notifica a la pantalla: "Ey, hay un vehículo nuevo". La pantalla se actualiza sin que tengas que hacer nada.

6.3 AppDatabase — La Base de Datos Misma

AppDatabase es la clase que "levanta" la base de datos SQLite en el móvil:

```
@Database(entities = {VehiculoEntity.class}, version = 1) public abstract
class AppDatabase extends RoomDatabase { public abstract VehiculoDao
vehiculoDao(); }
```

Aquí le dices a Room: "Voy a tener una tabla de Vehículos (entities), versión 1 de la BD".

6.4 VehiculoRepository — El Intermediario

Repository es una capa extra que aísla la base de datos de las pantallas. Las pantallas **nunca hablan directamente** con el DAO. Hablan con el Repository, y el Repository habla con el DAO.

```
@Singleton public class VehiculoRepository { private final VehiculoDao
vehiculoDao; private final ExecutorService executorService =
Executors.newFixedThreadPool(4); @Inject public
VehiculoRepository(VehiculoDao vehiculoDao) { this.vehiculoDao =
vehiculoDao; } public void insert(VehiculoEntity vehiculo)
{ executorService.execute(() -> { vehiculoDao.insert(vehiculo); }); } }
```

Nota lo importante: **ExecutorService**. La BD no se puede acceder en el hilo principal (el que ve el usuario). Si lo haces, la app se cuelga. Así que el Repository dice: "Hazlo en un hilo secundario".

7. HILT EN PROFUNDIDAD · Cómo Se Conectan las Piezas

Sin Hilt, si quisiera usar la BD en una pantalla, tendría que hacer esto manualmente:

```
// SIN HILT (la forma antigua y complicada) AppDatabase db =  
Room.databaseBuilder(context, AppDatabase.class,  
"reten_digital_db").build(); VehiculoDao dao = db.vehiculoDao();  
VehiculoRepository repo = new VehiculoRepository(dao); // Luego sí puedo  
usar el repo...
```

Con Hilt, es automático:

```
// CON HILT (la forma moderna y fácil) @Inject VehiculoRepository repo; //  
Ya está lista para usar. Hilt la creó automáticamente.
```

¿Cómo lo hace Hilt? A través del **AppModule**:

```
@Module @InstallIn(SingletonComponent.class) public class AppModule  
{ @Provides @Singleton public static AppDatabase  
provideAppDatabase( @ApplicationContext Context context) { return  
Room.databaseBuilder(context, AppDatabase.class,  
"reten_digital_db") .fallbackToDestructiveMigration() .build(); }  
@Provides public static VehiculoDao provideVehiculoDao( AppDatabase  
appDatabase) { return appDatabase.vehiculoDao(); } }
```

Esto le dice a Hilt: "Cuando alguien pida una AppDatabase, créala así. Cuando alguien pida un VehiculoDao, toma la AppDatabase y saca el DAO de ella".

¿Qué es @Singleton?

@Singleton significa que la BD se crea UNA SOLA VEZ cuando arranca la app, y todas las pantallas usan la misma instancia. Así no hay caos de múltiples BDs abiertas. Es como el "señor de los datos": uno solo para toda la app.

Línea crítica: RetenDigitalApp

Para que Hilt funcione, necesitas esta clase:

```
@HiltAndroidApp public class RetenDigitalApp extends Application { //  
    Vacía. Solo la anotación @HiltAndroidApp importa. // Esto le dice a Hilt:  
    "Arrancar aquí". }
```

Y en el AndroidManifest.xml, le dices a Android que use esta clase:

```
<application android:name=".RetenDigitalApp" ...>
```

8. LAS 5 PANTALLAS · Qué Hace Cada Una

8.1 MainActivity — El Login

Es la primera pantalla que ve el usuario. Tiene un campo de usuario, uno de contraseña y un botón "Iniciar Turno".

```
public class MainActivity extends AppCompatActivity { protected void onCreate(Bundle savedInstanceState) { super.onCreate(savedInstanceState); setContentView(R.layout.activity_main); MaterialButton btnIniciarTurno = findViewById(R.id.btnIniciarTurno); if (btnIniciarTurno != null) { btnIniciarTurno.setOnClickListener(v -> { // Cuando pulsa el botón, vamos al menú Intent intent = new Intent(MainActivity.this, MenuPrincipalActivity.class); startActivity(intent); finish(); // Cierra el login }); } } }
```

Nota: El login **no valida credenciales**. Solo navega al menú. Cualquiera puede entrar. Es la v1.0.

8.2 MenuPrincipalActivity — El Hub

Pantalla central con 4 botones:

- Vehículos → VehiculoListActivity
- Cuadrante → CuadranteActivity
- Inventario → InventarioActivity
- Intervenciones → IntervencionActivity

Cada botón crea un Intent (navegación Android) hacia la pantalla correspondiente:

```
MaterialButton btnVehiculos = findViewById(R.id.btnVehiculos); btnVehiculos.setOnClickListener(v -> { Intent intent = new Intent(MenuPrincipalActivity.this, VehiculoListActivity.class); startActivity(intent); });
```

8.3 VehiculoListActivity — La Flota

Muestra una lista de vehículos. Aquí es donde Room **realmente funciona** en tu app. Aunque en tu código actual la lista es visual (hardcodeada), la estructura está preparada para conectar un adapter y mostrar datos de la BD.

8.4 CuadranteActivity — Turnos

Muestra un calendario. El usuario puede ver qué días está de servicio. En tu código, el calendario está fijo en abril de 2026.

```
CalendarView calendarView = findViewById(R.id.calendarView); Calendar cal = Calendar.getInstance(); cal.set(2026, Calendar.APRIL, 16); calendarView.setDate(cal.getTimeInMillis(), false, true);
```

8.5 IntervencionActivity — El Incendio

La pantalla más compleja. Tiene:

- **Mapa Google Maps** con vista satélite
- **Botón GPS** que genera coordenadas UTM de Aragón
- **Campo de ubicación** donde se escriben las coordenadas
- **Botón Finalizar** que registra la intervención (actualmente solo muestra un Toast)

```
if (btnGPS != null) { btnGPS.setOnClickListener(v -> { String coordenadasAragonUTM = "30T X: 675200 Y: 4613500"; etUbicacion.setText(coordenadasAragonUTM); Toast.makeText(this, "Localización en Aragón fijada", Toast.LENGTH_SHORT).show(); }); }
```

El mapa se inicializa con el callback **onMapReady()**:

```
public void onMapReady(@NonNull GoogleMap googleMap) { mMap = googleMap; LatLng aragonPos = new LatLng(41.6488, -0.8891); // Zaragoza mMap.addMarker(new MarkerOptions().position(aragonPos).title("Punto de Intervención")); mMap.moveCamera(CameraUpdateFactory.newLatLngZoom(aragonPos, 12f)); mMap.setMapType(GoogleMap.MAP_TYPE_SATELLITE); // Satélite }
```

9. EL FLUJO COMPLETO · De Principio a Fin

Aquí explicamos qué pasa desde que el usuario abre la app hasta que hace una intervención:

1. Arranca el sistema

Android carga RetenDigitalApp (porque en el Manifest decimos `android:name=".RetenDigitalApp"`). La anotación `@HiltAndroidApp` activa Hilt.

2. Hilt crea la base de datos

Hilt mira el AppModule y dice: "Necesito crear una AppDatabase". Ejecuta la función `provideAppDatabase()` y crea la BD llamada "reten_digital_db" en el almacenamiento privado del móvil.

3. Se abre MainActivity (el login)

Android muestra la primera Activity definida en el Manifest como MAIN. El usuario ve un campo de usuario/contraseña y un botón "Iniciar Turno".

4. Usuario pulsa "Iniciar Turno"

El `onClick` del botón ejecuta: `startActivity(new Intent(..., MenuPrincipalActivity.class))`. Android navega a MenuPrincipalActivity. El `finish()` cierra MainActivity para que no pueda volver atrás.

5. Se abre MenuPrincipalActivity

El usuario ve 4 botones: Vehículos, Cuadrante, Inventario, Intervenciones.

6. Usuario pulsa "Intervenciones"

`startActivity(new Intent(..., IntervencionActivity.class))`. Android abre IntervencionActivity.

7. Se abre IntervencionActivity

`onCreate()` se ejecuta:

- Se buscan los componentes (botón GPS, mapa, campo de ubicación)
- Se inicializa Google Maps (`onMapReady` se ejecuta)
- El mapa muestra satélite de Zaragoza/Aragón

8. Usuario pulsa "Capturar GPS"

`onClick` del botón: `etUbicacion.setText("30T X: 675200 Y: 4613500")`. El campo se rellena con coordenadas UTM de ejemplo de Aragón.

9. Usuario pulsa "Finalizar Intervención"

`onClick`: `Toast.makeText(..., "Registro guardado en el servidor", ...).show(); finish()`.

NOTA: Esto muestra un Toast (cartel) pero NO guarda en BD todavía. La

implementación real requeriría:

- Crear IntervencionEntity
- Crear IntervencionDao
- Crear IntervencionRepository.insert()
- Llamar a repo.insert(new Intervencion(...))

10. Se cierra IntervencionActivity

finish() cierra esta pantalla y vuelve a MenuPrincipalActivity.

10. PREGUNTAS FRECUENTES SOBRE EL CÓDIGO

¿Por qué Room solo está implementado para Vehículos?

Porque implementar Room para todas las entidades (Vehículos, Cuadrante, Inventario, Intervenciones) requiere:

- 4 clases Entity
- 4 clases DAO
- 4 clases Repository
- Total: 12 clases nuevas

Por cuestión de tiempo, hiciste la "prueba de concepto" con Vehículos, y las demás pantallas quedaron como interfaces visuales. Es normal en desarrollo: haces un módulo completo, lo pruebas, y luego replicas el patrón en los demás.

¿Qué es ese "fallbackToDestructiveMigration()"?

En AppModule, hay esta línea:

```
.fallbackToDestructiveMigration().build();
```

Significa: "Si alguien cambia la estructura de la BD (por ejemplo, añade una columna), borra la BD vieja y crea una nueva". Esto es útil en desarrollo (los datos son ficticios), pero **peligroso en producción** (los datos reales desaparecen).

¿Las coordenadas UTM son reales?

No. En IntervencionActivity, cuando pulsa el botón GPS, hardcodea unas coordenadas de ejemplo:

```
String coordenadasAragonUTM = "30T X: 675200 Y: 4613500";
```

Estas son ficticias (aunque válidas para Aragón). Para capturar GPS real, necesitarías:

- FusedLocationProviderClient de Google Play Services
- Pedir permiso ACCESS_FINE_LOCATION en runtime
- Una función que convierta LatLng → UTM (conversión matemática complicada)

¿Retrofit está realmente integrado?

No. En build.gradle, está declarado:

```
implementation 'com.squareup.retrofit2:retrofit:2.9.0' implementation  
'com.squareup.retrofit2:converter-gson:2.9.0'
```

Pero en tu código Java, no hay ni un @GET ni un cliente Retrofit. Lo preparaste para versiones futuras (consultar Open-Meteo para el clima en la intervención).

¿Qué es un Intent?

Un Intent es el "mensaje" que Android usa para navegar entre pantallas. Cuando haces:

```
Intent intent = new Intent(MainActivity.this,  
    MenuPrincipalActivity.class); startActivity(intent);
```

Dices: "Crea un Intent de MainActivity a MenuPrincipalActivity, y ejecuta esa Activity". Android entenderá y abrirá la nueva pantalla.

¿Qué es un DAO?

DAO = Data Access Object. Es la interfaz entre tu código y la BD. En lugar de escribir SQL puro, usas métodos simples:

```
// Insertar dao.insert(vehiculo); // Leer List autos =  
dao.getAllVehiculos(); // Actualizar dao.update(vehiculo); // Borrar  
dao.delete(vehiculo);
```

¿Qué es un Entity?

Entity es una clase que representa una tabla en la BD. Cada propiedad de la clase es una columna de la tabla. Ejemplo:

```
@Entity(tableName = "tabla_vehiculos") public class VehiculoEntity
{ @PrimaryKey(autoGenerate = true) public int id; // Columna: id public
String matricula; // Columna: matricula public String marca; // Columna:
marca // ... }
```

¿Por qué Room en segundo plano (ExecutorService)?

Cuando escribes o lees de la BD, puede tardar unos milisegundos. Si lo haces en el hilo principal (donde corre la interfaz), la pantalla se congela. Por eso el Repository hace:

```
executorService.execute(() -> { vehiculoDao.insert(vehiculo); });
```

"Ejecuta la inserción en un hilo secundario". El hilo principal sigue respondiendo a los toques del usuario.

¿Y si quiero guardar una Intervención en la BD?

Tendrías que:

1. Crear **IntervencionEntity** (como VehiculoEntity, pero con campos de intervención)
2. Crear **IntervencionDao** (con insert, update, delete, getAllIntervenciones)
3. Crear **IntervencionRepository** (con métodos que llaman al DAO en segundo plano)
4. En IntervencionActivity, en lugar de Toast, llamarías a **repo.insert(new Intervencion(...))**
5. Room automáticamente guardaría en la BD

CONCLUSIÓN · Lo Que Has Logrado

Has creado una arquitectura profesional Android:

-  **Separación clara** entre UI, lógica y datos (MVVM)
-  **Base de datos local** offline-first con Room
-  **Inyección de dependencias** limpia con Hilt
-  **5 pantallas** navegables con Intent
-  **Google Maps integrado** con vista satélite
-  **Manejo de hilos** para no congelar la interfaz

Lo que falta para producción:

- Implementar Room para las demás entidades (Cuadrante, Inventario, Intervención)
- GPS real con FusedLocationProviderClient
- Conversión real de coordenadas LatLng → UTM
- Login con validación de credenciales
- Sincronización con servidor (Retrofit + API remota)
- Tests unitarios e integración
- Cifrado de BD (SQLCipher)
- Sistema de roles (permisos de usuario)

Todo esto está en el roadmap. La v1.0 es sólida y escalable.